task1 26.4.14

任务完成

- 完成了工作环境配置
- 读取音频 + 播放

```python
import librosa
import numpy as np
import matplotlib.pyplot as plt
import IPython.display as ipd

audio_path = "/Users/ye/Documents/musiclm-universe/notebook/dataset/657944__lilmati__skeleton-playing-piano-02.wav"

y, sr = librosa.load(audio_path, sr=24000)

print("Sample Rate:", sr)
print("Waveform shape:", y.shape)

ipd.display(ipd.Audio(y, rate=sr))
```

[2] 3.1s Python

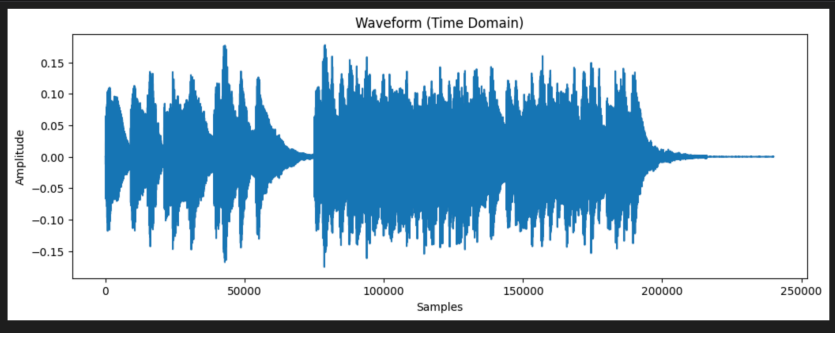
```
Sample Rate: 24000
Waveform shape: (240011,)
```

0:10 / 0:10

- 了解相关概念并生成时域波形图

```python
plt.figure(figsize=(12,4))
plt.plot(y)
plt.title("Waveform (Time Domain)")
plt.xlabel("Samples")
plt.ylabel("Amplitude")
plt.show()
```

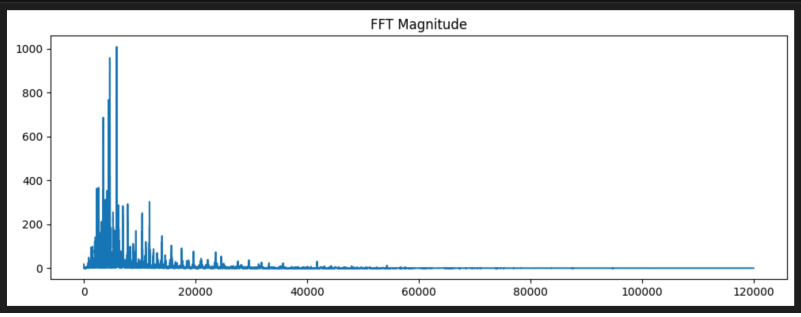
[3] 1.2s Python



- 使用FTT（快速傅立叶变换）将时域信号转换为频域
  傅里叶变换的核心思想：任何复杂信号，都可以分解成很多不同频率的正弦波叠加。也就是：复杂波形（时域） = 一堆不同频率的波加在一起（频域）

```python
fft = np.fft.fft(y)
magnitude = np.abs(fft)

plt.figure(figsize=(12,4))
plt.plot(magnitude[:len(magnitude)//2])
plt.title("FFT Magnitude")
plt.show()
```

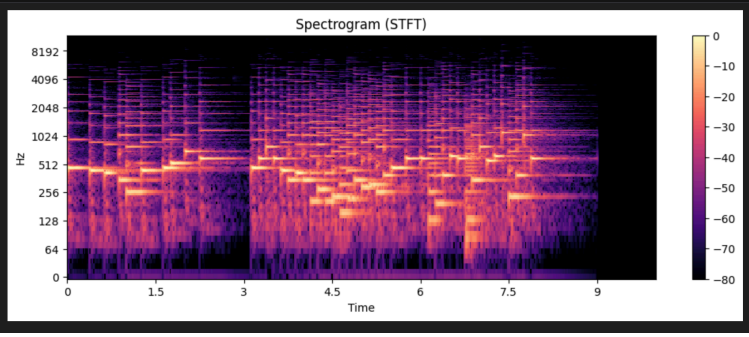
[7] 0.0s Python



- 了解声谱图（Spectrogram）概念
  为何使用STFT计算声谱图?
  普通 FFT 只能看到“整体有哪些频率“，但现实信号（比如说话、音乐）频率是随时间变化的
  所以使用STFT分段做FTT

```python
D = librosa.stft(y)
S_db = librosa.amplitude_to_db(np.abs(D), ref=np.max)

plt.figure(figsize=(12,4))
librosa.display.specshow(S_db, sr=sr, x_axis='time', y_axis='log')
plt.colorbar()
plt.title("Spectrogram (STFT)")
plt.show()
```

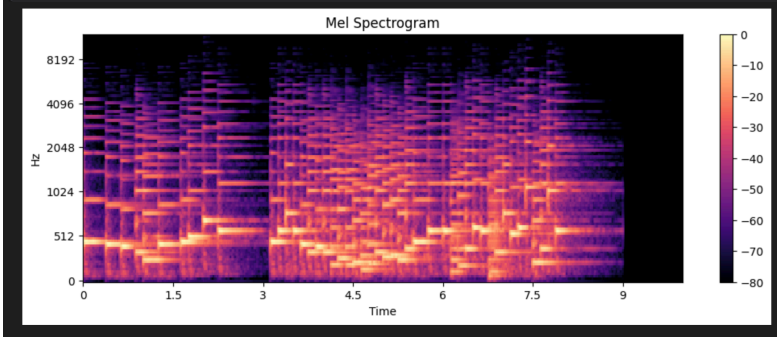
[8] 0.1s Python



- Mel 频谱图（Mel Spectrogram）
  Mel 刻度是用“人耳听觉刻度”表示的声谱图，一种“听觉刻度”
  与 **普通 Spectrogram** 的不同就是： 低频部分更“拉伸”，高频被压缩，这样就会更符合人耳听感

```python
mel = librosa.feature.melspectrogram(y=y, sr=sr, n_mels=128)
mel_db = librosa.power_to_db(mel, ref=np.max)

plt.figure(figsize=(12,4))
librosa.display.specshow(mel_db, sr=sr, x_axis='time', y_axis='mel')
plt.colorbar()
plt.title("Mel Spectrogram")
plt.show()
```

0.5s Python



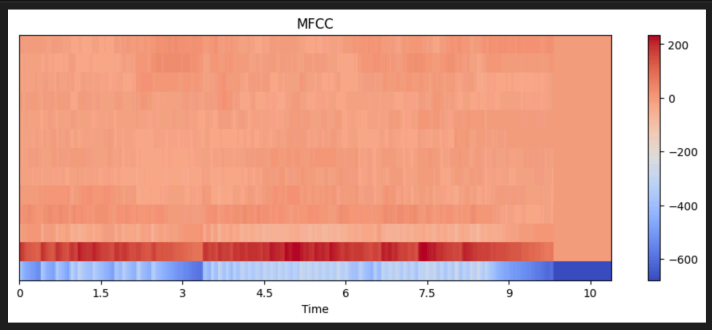
  Mel Spectrogram 很重要：它几乎是所有语音 AI 的标准输入
  可用于多种场景： 语音识别（ASR）、语音合成（TTS）、 音乐分类、说话人识别、语音大模型
  因为神经网络更容易从 Mel 表示中学到有意义的特征
- 了解MFCC（Mel 频率倒谱系数）相关概念，即从声音中提取“人耳感知特征”的一组数字特征
  MFCC（Mel Frequency Cepstral Coefficients）是语音识别里最经典、最常用的特征之一，
  其提取的是：**声音的整体频谱形**，而不是具体频率峰值

  Mel → Mel 刻度：用人耳听觉刻度
  Frequency → 频率：来自频域分析
  Cepstral → 倒谱：对频谱再做一次变换

Coefficients → 系数：最终输出的是一组数字

```python
mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13)

plt.figure(figsize=(12,4))
librosa.display.specshow(mfcc, x_axis='time')
plt.colorbar()
plt.title("MFCC")
plt.show()

print("MFCC shape:", mfcc.shape)
```

[10] ✓ 0.0s Python




```
MFCC shape: (13, 469)
```

- 了解MIDI符号表示（Musical Instrument Digital Interface）概念——一种音乐数据协议，用来描述“演奏动作”

MIDI具有Symbolic性质——用符号规则表示音乐结构，而不是连续信号

在 AI 里，Symbolic 音乐通常是： 音高（pitch）、起始时间（onset）、时值（duration）、力度（velocity）

```python
import pretty_midi

midi = pretty_midi.PrettyMIDI("/Users/ye/Documents/musiclm-universe/notebook/dataset/657944__lilmati__skeleton-playing-piano-02.mid")

for note in midi.instruments[0].notes[:10]:
    print("Pitch:", note.pitch,
          "Start:", note.start,
          "End:", note.end)
```

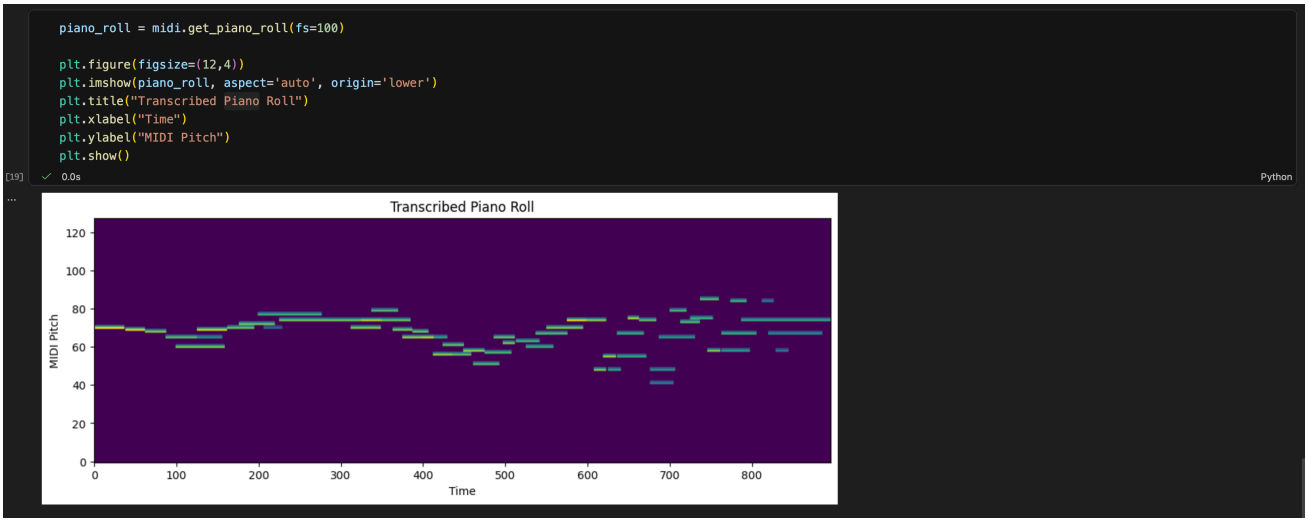
[18] ✓ 0.0s Python

```
Pitch: 70 Start: 0.011363636363636364 End: 0.3704545454545454
Pitch: 69 Start: 0.38409090909090091 End: 0.6272727272727272
Pitch: 68 Start: 0.6272727272727272 End: 0.8818181818181817
Pitch: 65 Start: 0.8704545454545454 End: 1.2545454545454544
Pitch: 65 Start: 1.2545454545454544 End: 1.5681818181818181
Pitch: 60 Start: 0.9977272727272727 End: 1.5909090909090908
Pitch: 69 Start: 1.2545454545454544 End: 1.625
Pitch: 70 Start: 1.625 End: 1.95
Pitch: 72 Start: 1.7636363636363634 End: 2.2068181818181816
Pitch: 70 Start: 2.068181818181818 End: 2.3
```

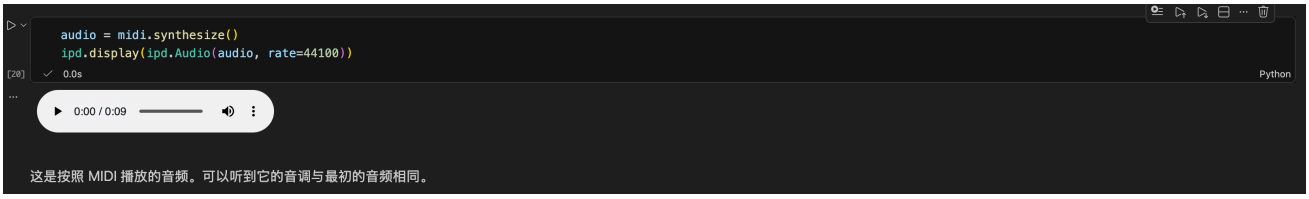
```markdown
从输出中可以看到：

比如第二行：

- 在 0.384 秒时按下一个音高为 69 的音（对应的音 应该是 A4，中央A）
- 在 0.627 秒时松开
- 持续时间 ≈ 0.243 秒

---
```

markdown

- MIDI Piano Roll



- MIDI Playback



学习心得

学习了音频信号处理的基础知识，主要包括时域到频域的转换（FFT）、Mel频谱图的原理及其在语音AI中的重要性，以及MFCC特征提取和MIDI音乐符号化表示。这些概念相互关联，构成了理解语音和音乐AI的基石。Mel频谱图作为标准输入的特点让我印象较为深刻，后续我会尝试实际应用这些知识。